UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

CMSC 12 — Fundamentals of Programming 2
Second Semester AY 2025-2026

MACHINE PROBLEM

The Last Stand

Technical Report

Submitted by:

Member 1 Program Institution City of Institution	Dizahlene Catenza BS in Computer Science University of the Philippines Tacloban Tacloban, Philippines	Member 2 Program Institution City of Institution	Joshua FildamChristopher Chu BS in Computer Science University of the Philippines Tacloban Tacloban, Philippines
Member 3 Program Institution City of Institution	Allyssa May Cristino BS in Computer Science University of the Philippines Tacloban Tacloban, Philippines	Member 4 Program Institution City of Institution	Matt Lenard Laraga BS in Computer Science University of the Philippines Tacloban Tacloban, Philippines

Instructor:

Samson Jr. D. Rollo
Bachelor of Science in Computer Science

20 May 2026

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

A. Rationale and Background

In this project, we implemented a 2D wave top-down shooter game inspired by the mini-game Journey of the Prairie King from Stardew Valley, set in a medieval fantasy world, but with an original title: The Last Stand. In this game, the player controls a lone knight defending a besieged fortress against endless waves of enemies that close in from all sides. As enemies fall, they drop relics that the knight can collect to upgrade their abilities and push through each siege. The fortress will not fall while the knight still stands. The game was built entirely in Java using AWT/Swing for rendering and input, and serves as a practical application of core object-oriented programming principles — inheritance, polymorphism, and encapsulation — alongside concurrent threading, 2D graphics, basic physics, and file-based game

B. Objectives

- Applied OOP concepts such as inheritance, proper encapsulation, and polymorphism in implementing the game components. The class hierarchy demonstrates these principles across all interactive objects. Enemy variants all inherit shared pathfinding and animation logic from the abstract `Enemy` class while overriding only what differs — speed, health, and sprite sets.
- Applied 2D Graphics by building a rendering pipeline inside a Swing `JPanel` that composites a background tile layer, game objects with Y-sorted depth ordering, a foreground overlay, and a HUD layer, all within a single `paintComponent` cycle. Sprite animation is driven by frame-indexed image strips advanced per game tick.
- Implemented concurrent threads to keep the game running at a stable 60 FPS. A dedicated `GameThread` runs a fixed-timestep loop decoupled from Swing's Event Dispatch Thread, preventing UI freezes during intensive game logic.
- Applied basic physics such as normalized velocity vectors for 8-directional player and bullet movement, subpixel accumulation to eliminate drift, and rectangle-based collision detection between all game objects and wave obstacles.
- Applied a system that tracks the current wave, life count of the player character, the number of active enemies, and overall game progress — which can be saved to disk and restored in a future session via the Continue Game feature.

C. Tools and Technologies

- Language and Runtime

  Java 17 (OpenJDK) — primary language and runtime environment

- Java Standard Library APIs Used

`javax.swing` / `java.awt` — window management, 2D rendering (`JPanel`, `Graphics`, `Image`), and Swing UI components (`JLayeredPane`, `CardLayout`, `JButton`) - `javax.sound.sampled` — background music looping and sound effect playback via `Clip` and `FloatControl` - `java.io` / `java.nio` — save file creation, reading, and deletion (`FileWriter`, `PrintWriter`, `Scanner`, `Files`, `Paths`) - `java.util` — core data structures (`CopyOnWriteArrayList`, `PriorityQueue`, `ArrayDeque`, `HashMap`, `HashSet`) and utilities (`Random`, `Collections`)

- Development Tools

  VS Code — primary code editor

- Asset Creation Tools

  Clip Studio Paint — all in-game sprite sheets, obstacle art, and background images were created from scratch by the team - Canva — UI panel and interface assets

- Fonts
  - MainMenuTitle — custom font designed by Matt Lenard Laraga
  - Brick Sans by Fatfly Design
  - Biski by Jipatype
- Audio

  Background music and sound effects sourced from YouTube Music and Pixabay (royalty-free)

D. Functional Requirements (Gameplay Dynamics)

Player & Combat

The player uses arrow keys for 8-directional movement with normalized diagonal speed and WASD to shoot bullets in eight corresponding directions. A subpixel remainder ensures precise movement. The base fire rate has a 500ms cooldown, which is reduced to 200ms with a powerup. Each shot displays a temporary attack sprite overlay.

Enemies

All enemies utilize A* pathfinding on a grid, rate-limited to balance performance. A stuck-detection system nudges enemies after 30 frames of inactivity. Enemies alternate between a WALK state and a directional 6-frame ATTACK animation, with a 50% hit chance. Hits deduct a life (with a contact cooldown), while misses display a fading "MISS" image.

There are five unique enemy types, each with specific attributes:

- BasicEnemy: Standard stats.
- SpeedyEnemy: Fast-moving (5 speed).
- TankyEnemy: More durable (2 health).
- BossEnemy: High health, twice the normal size, and periodically spawns minions until low on health.
- BossMinionEnemies: The spawned minions.

The probability of special enemy types (Speedy, Tanky) spawning increases with each wave.

Progression & Levels

The game spans 5 waves. The enemy count per wave is calculated with a specific formula, and the time between forced waves slightly increases later. The final BossEnemy appears on the last wave. A wave is completed when all enemies are defeated.

Powerups & Obstacles

Enemies have a 10% chance to drop a powerup on death, provided one isn't already present on the field. The three powerups are:

- FireRatePowerup: Reduced fire rate for 5 seconds.
- MovementSpeedPowerup: Increased player speed for 10 seconds.
- HealPowerUp: Instantly restores one life.

Each wave features static typed obstacles with hitboxes that sometimes allow walking behind them. The pathfinding grid accommodates these obstacle bounds to prevent clipping.

Game State & Systems

- Win/Loss: The player loses when lives reach 0, showing a screen to respawn (restart the wavel) or return to the menu. The game is won by defeating the BossEnemy. In both scenarios, the active save file is deleted.
- Saving & Loading: A simple save system writes the current state (wave, lives, position, and spawn rate) to a text file. The Continue button on the main menu is only active when a valid save file exists.
- Audio: A Singleton-patterned system manages looping background music and on-demand SFX, automatically releasing resources after sounds finish playing.

E. Methodology

Asset Design

- Custom Sprites: Created using Clip Studio Paint and Canva. Includes player/enemy animations (8 directions), obstacles, powerups, backgrounds, and UI.
- Loading: Asset strips use a prefix + frameNumber + .png naming convention. The Entity.loadStrip() utility loads frames once into static arrays to conserve memory.

OOP Class Hierarchy

The architecture uses inheritance to isolate behaviors and maximize code reuse:

- Game Objects: GameObject → Entity (Player, Enemy subtypes, Bullet), Obstacle, and abstract Powerup subtypes.
- UI Panels: JPanel → MainPanel (uses CardLayout) and OverlayPanel (Pause, Game Over, Win, etc.).
- Layering & Navigation: Overlays sit on a JLayeredPane's MODAL_LAYER above the game. Panel switching is decoupled using a Consumer<String> lambda.

Game Loop

- Pattern: Uses a fixed-timestep with lag accumulation (targeting 60 FPS) so logic updates uniformly regardless of render frame rates.
- Catch-up Cap: Lag catch-up is capped at 5 framesto prevent performance death spirals after system pauses.
- Thread Management: Runs on a dedicated "GameThread" that supports clean pausing (wait/notifyAll) and stopping.

Rendering Pipeline

Each frame composites layers in the following order to ensure correct visual depth:

a. Grass background
b. Bullets
c. Enemies (with floating "MISS" text)
d. Active powerup icon
e. Obstacles & Player: Sorted by Y-position (y + height) for realistic overlapping
f. Grass foreground overlay
g. HUD (Player lives and wave counter)
h. Boss health bar (rendered only during boss fights)

F. Results

Figure 1 — Main Menu screen with New Game, Continue, Exit, About, and Credits buttons

Figure 2 — Wave 1 gameplay showing the player, enemies, and obstacle layout

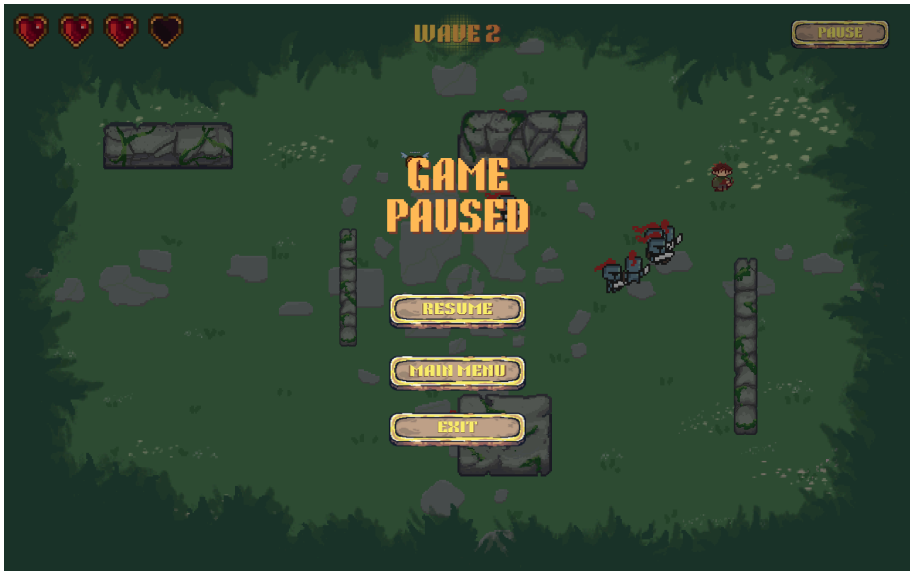

Figure 3 — Pause menu overlay

Figure 4 — Win screen after defeating the boss

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

Figure 5 — Game Over screen with Respawn and Main Menu options

G. Problems/Bugs

Pathfinding & Spawning

- Enemy Clustering & Freezing: Enemies can overlap in the same cell, blocking those behind them. If the player is completely walled off, A* returns `null` and enemies freeze.
- Bad Spawn Coordinates: The player's safe-respawn logic can fail on dense maps and trap them inside obstacles. Similarly, boss minions spawn in a 3×3 grid without checking for walls, occasionally trapping them inside geometry.

Collision & Saving

- Wall Sticking: Collision reverts both X and Y axes simultaneously on a hit. This causes the player to stick to walls instead of sliding smoothly along them.
- Bullet Tunneling: Fast bullets do not perform sub-step collision checks and can skip through thin obstacles entirely.
- Save File Flaws: Stored player X/Y coordinates are ignored on load (the player resets to safe-start positions instead). Additionally, forcing the game to quit mid-save corrupts the file.

Audio, UI & Code Cleanup

- Audio Crashes: Playing too many sound effects at once opens too many `Clip` instances, exhausting system lines and causing `LineUnavailableException`. There is also no in-game volume menu.
- UI & Code Bugs: The `CreditsPanel` closes accidentally due to an overlapping mouse listener.

Lastly, a debug cheat key (`K` instantly kills the boss) and a typo method (`getSoawnRate()`) still remain in the production code.